In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import kruskal

# Import the dataset
df=pd.read_csv("311_Service_Requests_from_2010_to_Present.csv")
df.head()
```

```
C:\Users\sanket\AppData\Local\Temp\ipykernel_25444\787832099.py:4: DtypeWarning: Columns (48,49) have mixed types. Specify dtyp
e option on import or set low_memory=False.
  df=pd.read_csv("311_Service_Requests_from_2010_to_Present.csv")
```

Out[1]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	Location Type	Incident Zip	Incident Address	...	Bridge Highway Name
0	32310363	12/31/2015 11:59:45 PM	01/01/2016 12:55:15 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party	Street/Sidewalk	10034.0	71 VERMILYEA AVENUE	...	NaN
1	32309934	12/31/2015 11:59:44 PM	01/01/2016 01:26:57 AM	NYPD	New York City Police Department	Blocked Driveway	No Access	Street/Sidewalk	11105.0	27-07 23 AVENUE	...	NaN
2	32309159	12/31/2015 11:59:29 PM	01/01/2016 04:51:03 AM	NYPD	New York City Police Department	Blocked Driveway	No Access	Street/Sidewalk	10458.0	2897 VALENTINE AVENUE	...	NaN
3	32305098	12/31/2015 11:57:46 PM	01/01/2016 07:43:13 AM	NYPD	New York City Police Department	Illegal Parking	Commercial Overnight Parking	Street/Sidewalk	10461.0	2940 BAISLEY AVENUE	...	NaN
4	32306529	12/31/2015 11:56:58 PM	01/01/2016 03:24:42 AM	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk	Street/Sidewalk	11373.0	87-14 57 ROAD	...	NaN

5 rows × 53 columns

In [2]:
```
# Print the columns of the DataFrame & Identify the shape of the dataset
print(df.columns)
print(df.shape)
```

```
Index(['Unique Key', 'Created Date', 'Closed Date', 'Agency', 'Agency Name',
       'Complaint Type', 'Descriptor', 'Location Type', 'Incident Zip',
       'Incident Address', 'Street Name', 'Cross Street 1', 'Cross Street 2',
       'Intersection Street 1', 'Intersection Street 2', 'Address Type',
       'City', 'Landmark', 'Facility Type', 'Status', 'Due Date',
       'Resolution Description', 'Resolution Action Updated Date',
       'Community Board', 'Borough', 'X Coordinate (State Plane)',
       'Y Coordinate (State Plane)', 'Park Facility Name', 'Park Borough',
       'School Name', 'School Number', 'School Region', 'School Code',
       'School Phone Number', 'School Address', 'School City', 'School State',
       'School Zip', 'School Not Found', 'School or Citywide Complaint',
       'Vehicle Type', 'Taxi Company Borough', 'Taxi Pick Up Location',
       'Bridge Highway Name', 'Bridge Highway Direction', 'Road Ramp',
       'Bridge Highway Segment', 'Garage Lot Name', 'Ferry Direction',
       'Ferry Terminal Name', 'Latitude', 'Longitude', 'Location'],
      dtype='object')
(364558, 53)
```

In [3]:
```
# Identify the variables with null values
df.isnull().sum()
```

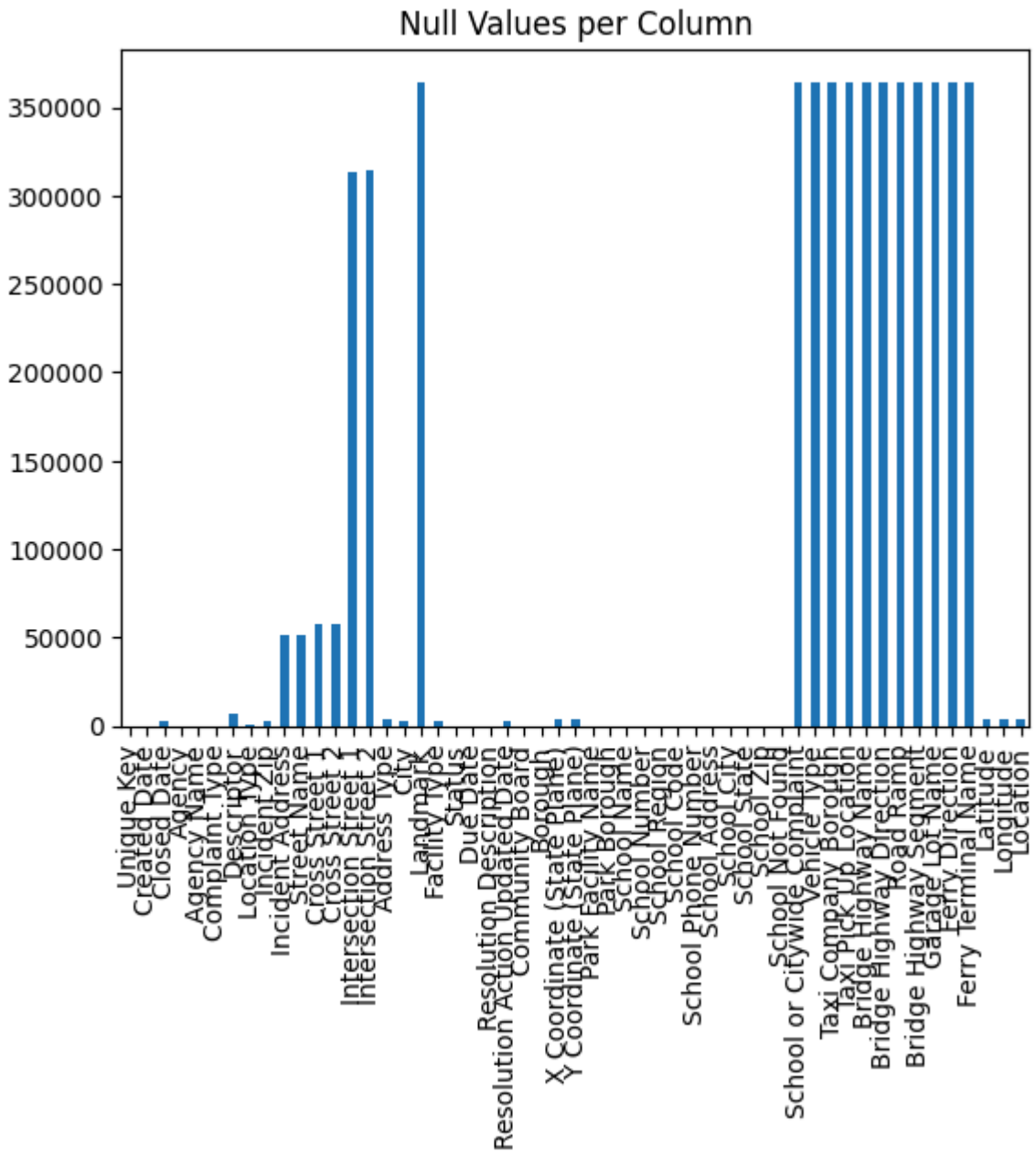
Out[3]:

Unique Key	0
Created Date	0
Closed Date	2381
Agency	0
Agency Name	0
Complaint Type	0
Descriptor	6501
Location Type	133
Incident Zip	2998
Incident Address	51699
Street Name	51699
Cross Street 1	57188
Cross Street 2	57805
Intersection Street 1	313438
Intersection Street 2	314046
Address Type	3252
City	2997
Landmark	364183
Facility Type	2389
Status	0
Due Date	3
Resolution Description	0
Resolution Action Updated Date	2402
Community Board	0
Borough	0
X Coordinate (State Plane)	4030
Y Coordinate (State Plane)	4030
Park Facility Name	0
Park Borough	0
School Name	0
School Number	0
School Region	1
School Code	1
School Phone Number	0
School Address	0
School City	0
School State	0
School Zip	1
School Not Found	0
School or Citywide Complaint	364558

```
Vehicle Type                     364558
Taxi Company Borough             364558
Taxi Pick Up Location            364558
Bridge Highway Name              364261
Bridge Highway Direction         364261
Road Ramp                        364296
Bridge Highway Segment           364296
Garage Lot Name                  364558
Ferry Direction                  364557
Ferry Terminal Name              364556
Latitude                           4030
Longitude                          4030
Location                           4030
dtype: int64
```

In [4]:

```python
# Draw a frequency plot to show the number of null values in each column of the DataFrame
df.isnull().sum().plot(kind='bar')
plt.title("Null Values per Column")
plt.show()
```



In [5]:
```
# Remove the records whose Closed Date values are null
```

```
df = df[df['Closed Date'].notnull()]
```

In [6]:
```
# Analyze the date column, and remove entries that have an incorrect timeline
df = df[df['Closed Date'] >= df['Created Date']]
```

In [7]:
```
# 1Calculate the time elapsed in closed and creation date
df['Created Date'] = pd.to_datetime(df['Created Date'])
df['Closed Date'] = pd.to_datetime(df['Closed Date'])
df['Elapse Time'] = df['Closed Date'] - df['Created Date']
print(df['Elapse Time'])
```

```
C:\Users\sanket\AppData\Local\Temp\ipykernel_25444\959916214.py:1: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  df['Created Date'] = pd.to_datetime(df['Created Date'])
C:\Users\sanket\AppData\Local\Temp\ipykernel_25444\959916214.py:2: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  df['Closed Date'] = pd.to_datetime(df['Closed Date'])
26       0 days 00:26:50
36       0 days 00:18:45
38       0 days 00:29:28
40       0 days 00:04:55
41       0 days 00:04:01
               ...
364538   0 days 00:20:09
364545   0 days 00:33:26
364547   0 days 00:41:04
364551   0 days 00:37:39
364555   0 days 00:19:03
Name: Elapse Time, Length: 294769, dtype: timedelta64[ns]
```

In [8]:
```
# Convert the calculated date to seconds to get a better representation
df['Elapse Time (sec)'] = df['Elapse Time'].dt.total_seconds()
print(df['Elapse Time (sec)'])
```

```
26        1610.0
36        1125.0
38        1768.0
40         295.0
41         241.0
           ...
364538    1209.0
364545    2006.0
364547    2464.0
364551    2259.0
364555    1143.0
Name: Elapse Time (sec), Length: 294769, dtype: float64
```

In [9]:
```
# View the descriptive statistics for the newly created column
df['Elapse Time (sec)'].describe()
```

Out[9]:
```
count    2.947690e+05
mean     1.466177e+04
std      2.280933e+04
min      6.100000e+01
25%      3.950000e+03
50%      8.409000e+03
75%      1.709700e+04
max      2.134342e+06
Name: Elapse Time (sec), dtype: float64
```

In [10]:
```
# Check the number of null values in the Complaint_Type and City columns
df[['Complaint Type', 'City']].isnull().sum()
```

Out[10]:
```
Complaint Type      0
City              488
dtype: int64
```

In [11]:
```
# Impute the NA value with Unknown City
df['City'].fillna("Unknown City", inplace=True)
df['City'].isnull().sum()
```

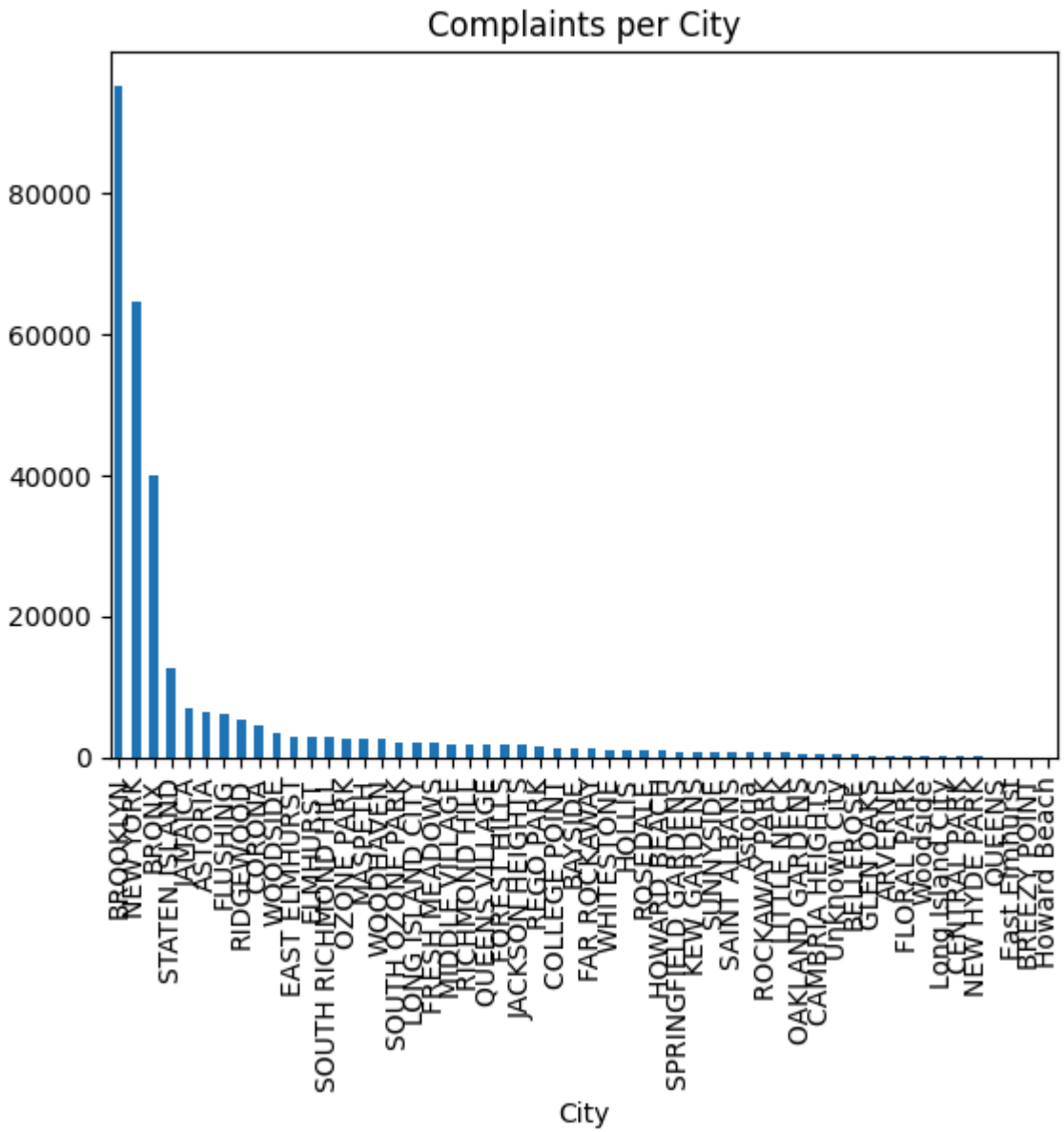
```
C:\Users\sanket\AppData\Local\Temp\ipykernel_25444\1038464959.py:1: FutureWarning: A value is trying to be set on a copy of a D
ataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are set
ting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = d
f[col].method(value) instead, to perform the operation inplace on the original object.


  df['City'].fillna("Unknown City", inplace=True)
```

Out[11]:

```
np.int64(0)
```

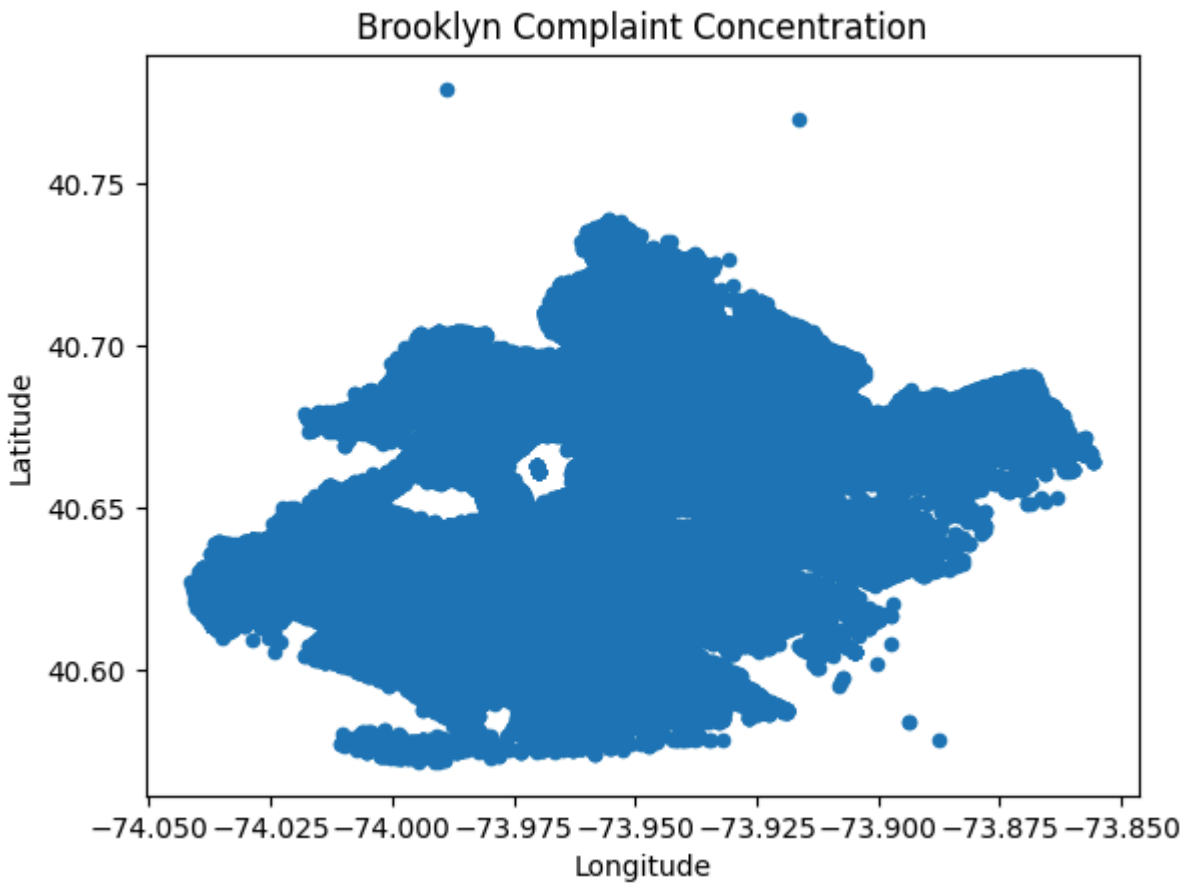
In [12]:

```
# Draw a frequency plot for the complaints in each city
df['City'].value_counts().plot(kind='bar')
plt.title("Complaints per City")
plt.show()
```

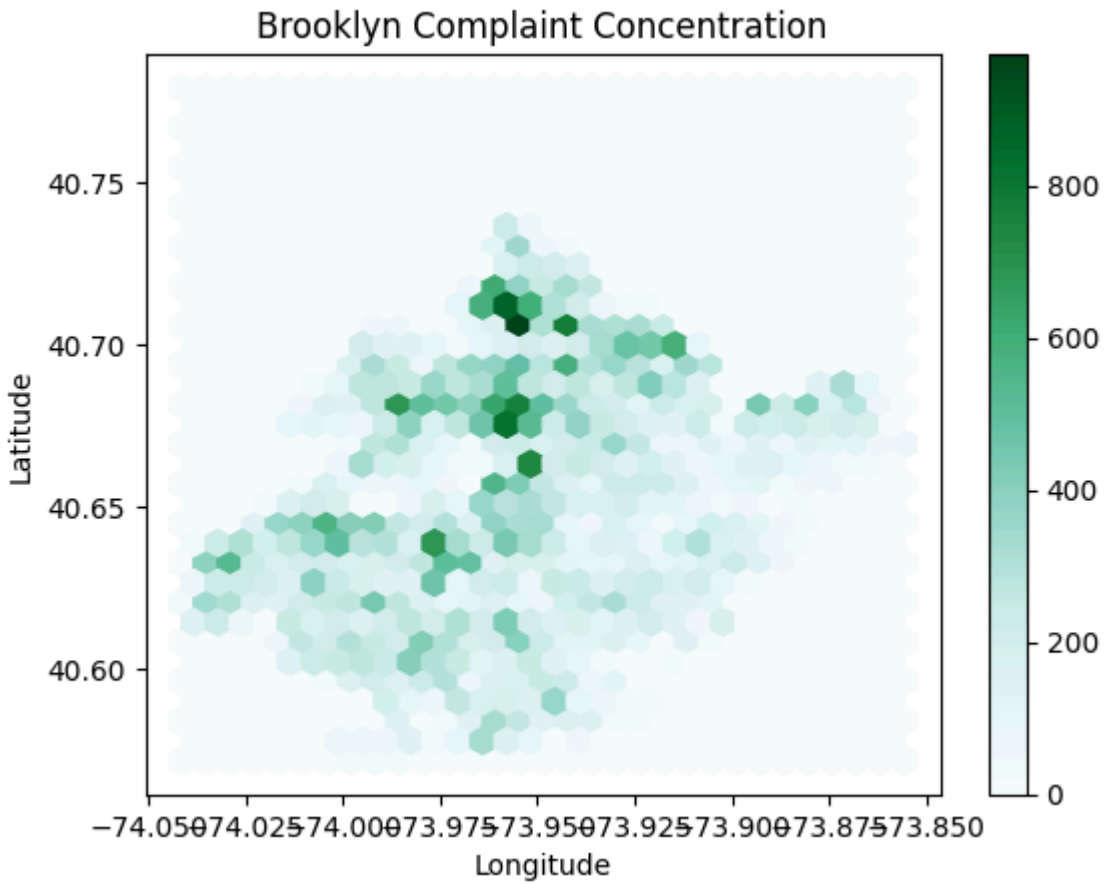


In [13]:
```
# Create a scatter and hexbin plot of the concentration of complaints across Brooklyn
brooklyn_com = df[df['City'] == 'BROOKLYN']
brooklyn_com.plot.scatter(x='Longitude', y='Latitude')
```

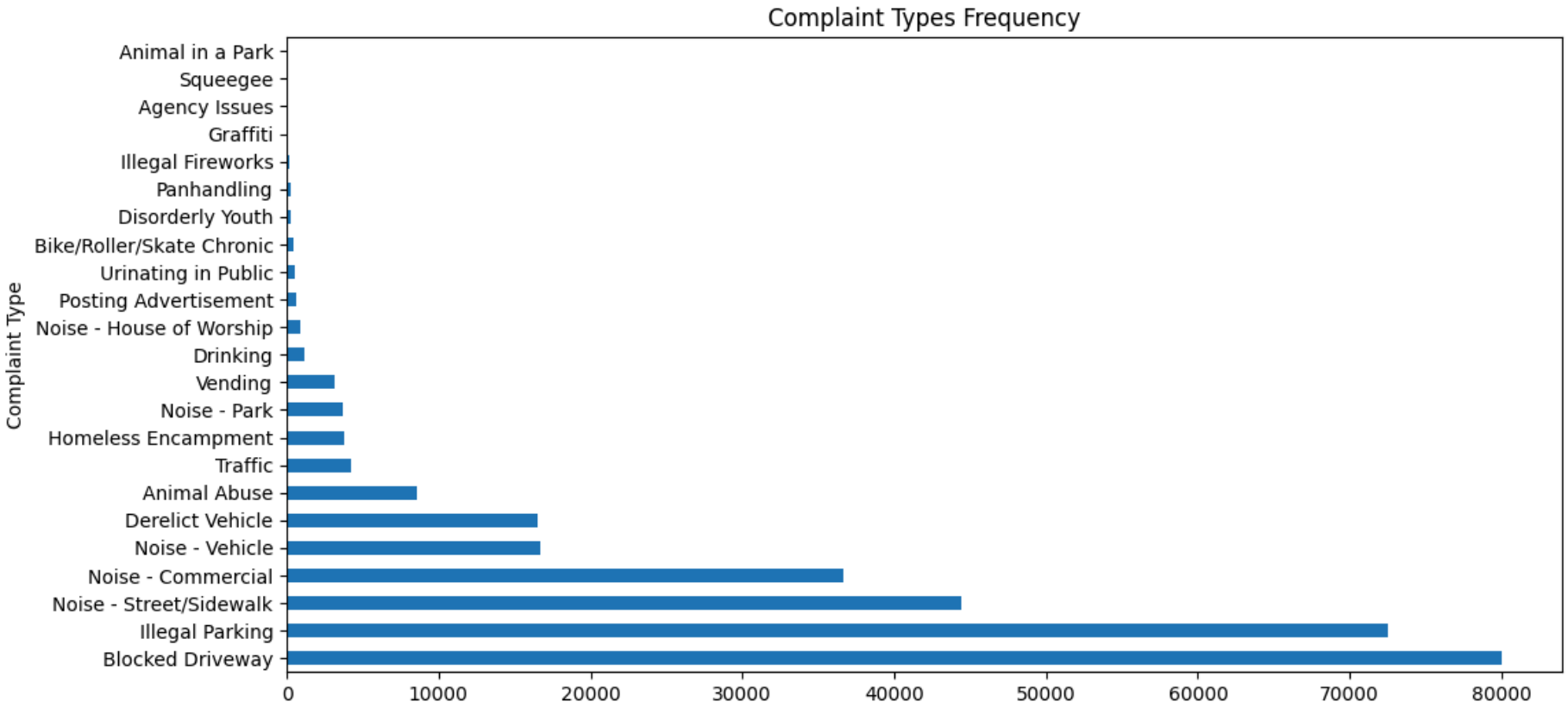
```
plt.title("Brooklyn Complaint Concentration")
plt.show()
```



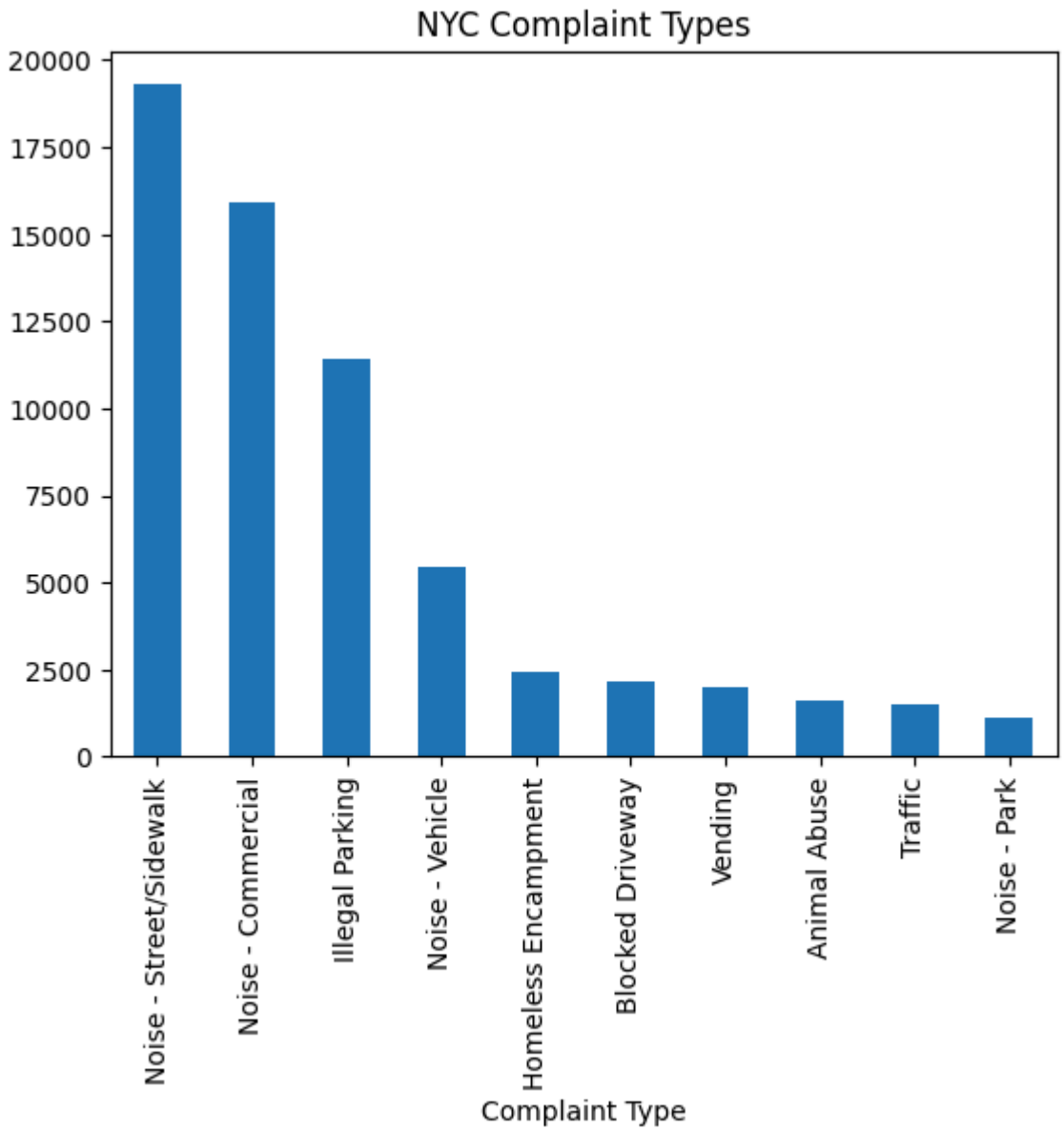
In [14]:
```
brooklyn_com = df[df['City'] == 'BROOKLYN']
brooklyn_com.plot.hexbin(x='Longitude', y='Latitude',gridsize=30)
plt.title("Brooklyn Complaint Concentration")
plt.show()
```



In [15]:
```
# Plot a bar graph to show the types of complaints
df['Complaint Type'].value_counts().plot(kind='barh', figsize=(12,6))
plt.title("Complaint Types Frequency")
plt.show()
```



In [16]:
```
# Check the frequency of various types of complaints for New York City
nyc_com = df[df['City'] == 'NEW YORK']
nyc_com['Complaint Type'].value_counts().head(10).plot(kind='bar')
plt.title("NYC Complaint Types")
plt.show()
```



In [17]:
```
# Find the top 10 complaint types
top_10 = df['Complaint Type'].value_counts().head(10)
print(top_10)
```

```
Complaint Type
Blocked Driveway           79984
Illegal Parking            72531
Noise - Street/Sidewalk    44442
Noise - Commercial         36634
Noise - Vehicle            16727
Derelict Vehicle           16512
Animal Abuse                8538
Traffic                     4276
Homeless Encampment         3802
Noise - Park                3658
Name: count, dtype: int64
```

In [18]:

```
# Display the various types of complaints in each city
df_new = pd.crosstab(df['Complaint Type'], df['City'])
df_new
```

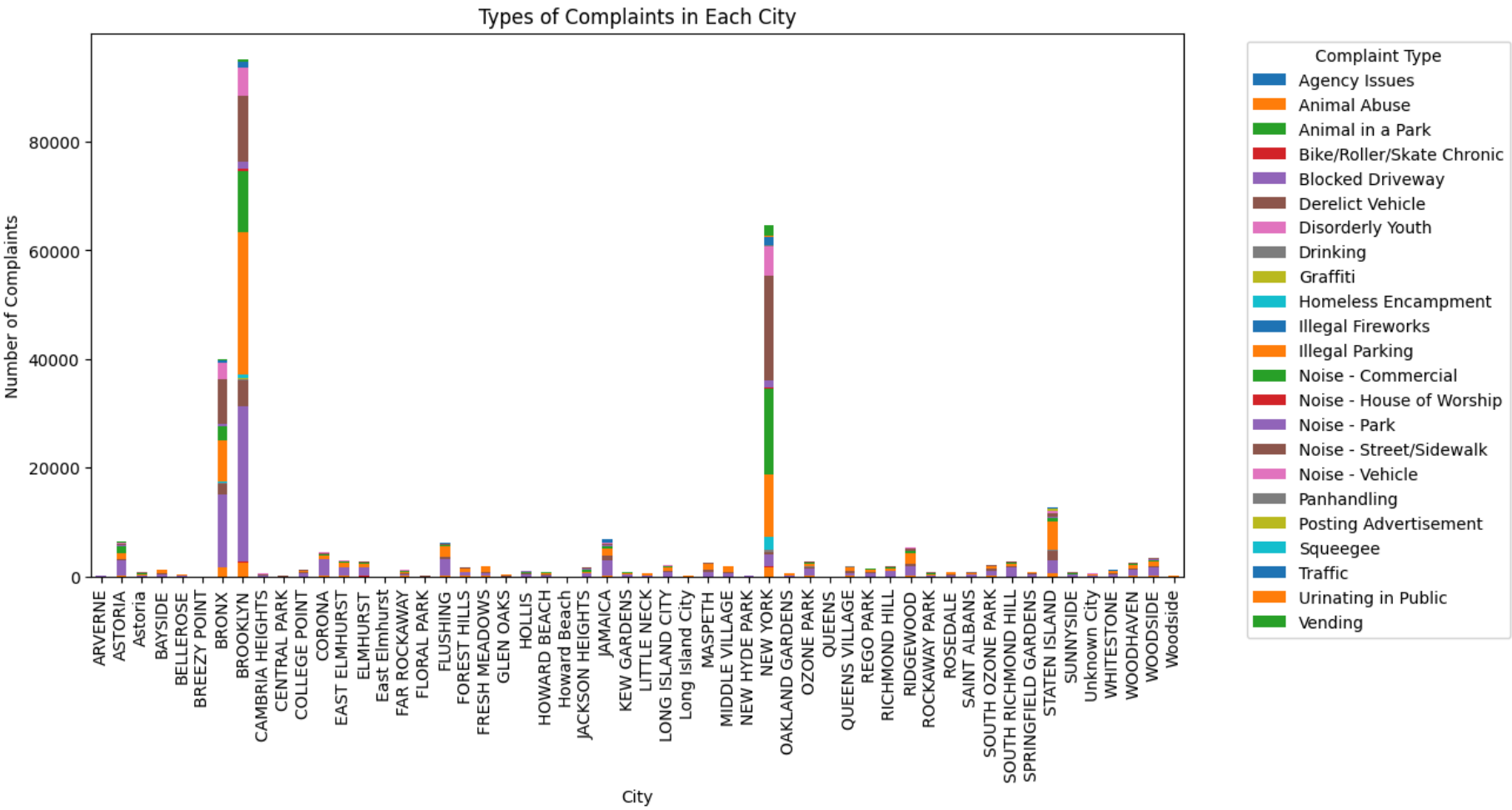
Out[18]:

City	ARVERNE	ASTORIA	Astoria	BAYSIDE	BELLEROSE	BREEZY POINT	BRONX	BROOKLYN	CAMBRIA HEIGHTS	CENTRAL PARK	...	SOUTH OZONE PARK	RI
Complaint Type													
Agency Issues	0	0	0	0	0	0	0	0	0	0	...	0	
Animal Abuse	41	131	0	47	13	2	1567	2582	13	0	...	58	
Animal in a Park	0	0	0	0	0	0	0	0	0	0	...	0	
Bike/Roller/Skate Chronic	0	15	0	0	0	0	20	100	0	0	...	1	
Blocked Driveway	40	2720	132	413	119	3	13546	28628	141	0	...	974	
Derelict Vehicle	23	323	9	179	102	1	1858	4747	123	0	...	335	
Disorderly Youth	2	4	0	2	2	0	62	70	0	0	...	2	
Drinking	1	38	0	1	1	1	177	241	0	0	...	13	
Graffiti	1	4	0	1	0	0	10	45	0	0	...	1	
Homeless Encampment	2	19	0	2	1	0	200	732	4	0	...	5	
Illegal Fireworks	0	4	0	0	1	0	21	51	1	0	...	1	
Illegal Parking	47	1037	230	536	104	14	7590	26026	85	5	...	497	
Noise - Commercial	2	1333	251	38	33	3	2535	11361	12	0	...	63	
Noise - House of Worship	14	17	0	2	1	0	77	333	2	0	...	4	
Noise - Park	2	56	0	4	1	0	507	1378	0	0	...	4	
Noise - Street/Sidewalk	27	351	123	16	11	1	8017	12132	27	100	...	95	

City	ARVERNE	ASTORIA	Astoria	BAYSIDE	BELLEROSE	BREEZY POINT	BRONX	BROOKLYN	CAMBRIA HEIGHTS	CENTRAL PARK	...	SOUTH OZONE PARK	RI
Complaint Type													
Noise - Vehicle	9	208	0	21	9	1	3106	5104	86	0	...	86	
Panhandling	1	2	0	0	0	0	16	37	0	0	...	0	
Posting Advertisement	0	3	0	0	1	0	16	47	0	0	...	1	
Squeegee	0	0	0	0	0	0	0	0	0	0	...	0	
Traffic	1	42	0	8	8	0	345	1003	7	0	...	24	
Urinating in Public	1	8	0	0	0	0	44	125	0	0	...	2	
Vending	1	39	0	2	0	0	324	451	0	0	...	3	

23 rows × 54 columns

In [19]:
```
# Create a DataFrame, df_new, which contains cities as columns and complaint types in rows
# Draw another chart that shows the types of complaints in each city in a single chart, where different colors show the
different types of complaints
df_new=pd.crosstab(df['City'], df['Complaint Type'])
df_new.plot(kind='bar', stacked=True, figsize=(12,6))
plt.title("Types of Complaints in Each City")
plt.xlabel("City")
plt.ylabel("Number of Complaints")
plt.legend(title="Complaint Type", bbox_to_anchor=(1.05, 1))
plt.show()
```

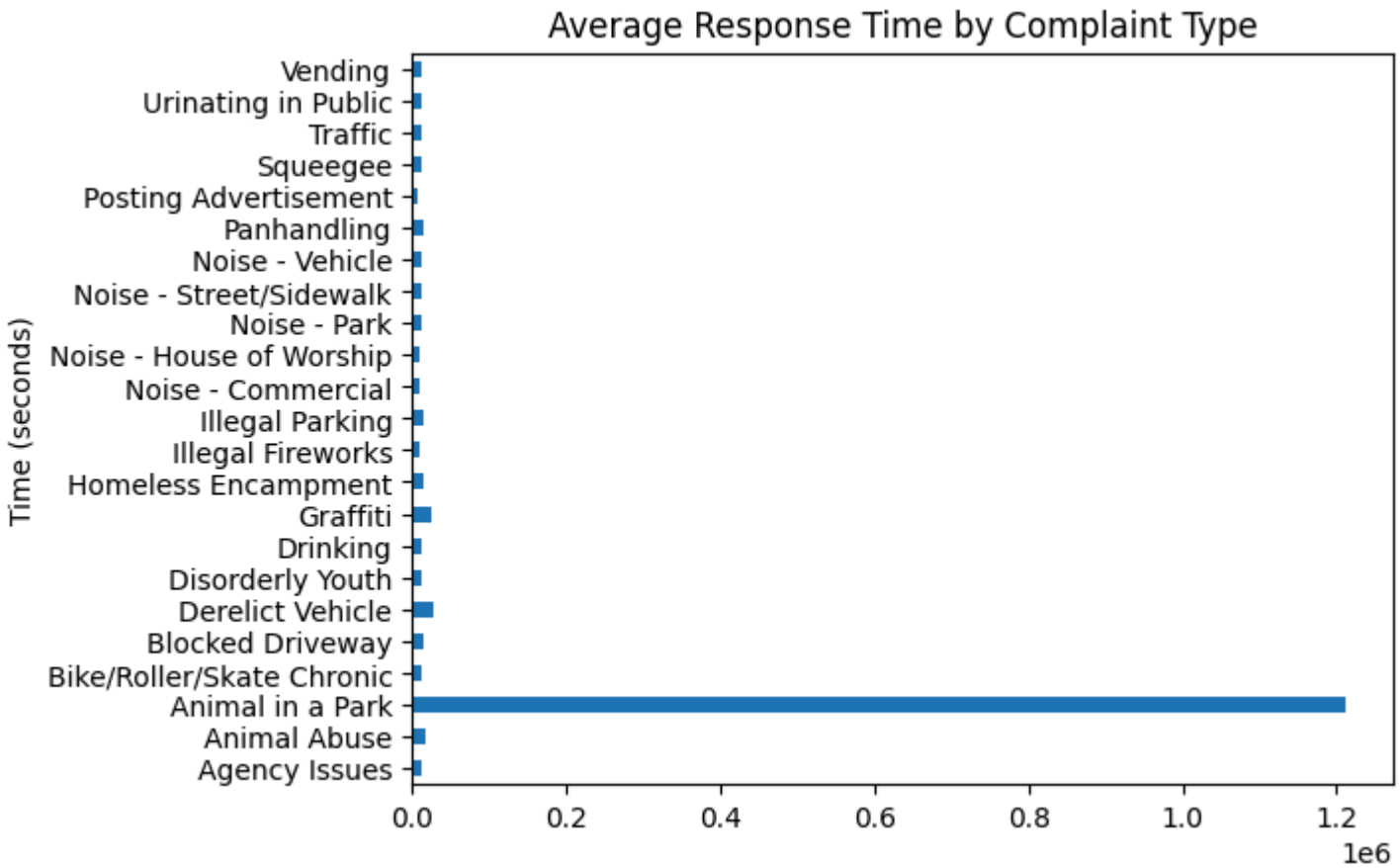


In [20]:
```
# Sort the complaint types based on the average Request_Closing_Timegrouping them for different locations
grouped = df.groupby(['City', 'Complaint Type'])['Elapse Time (sec)'].mean()
sorted_data = grouped.sort_values(ascending=False)
print(sorted_data)
```

```
City                 Complaint Type
QUEENS               Animal in a Park             1212634.0
QUEENS VILLAGE       Graffiti                      191995.0
BELLEROSE            Homeless Encampment           140930.0
SOUTH RICHMOND HILL  Bike/Roller/Skate Chronic     111277.0
OAKLAND GARDENS      Homeless Encampment           103156.0
                                                     ...
SUNNYSIDE            Urinating in Public              985.0
EAST ELMHURST        Bike/Roller/Skate Chronic        908.0
RIDGEWOOD            Posting Advertisement            895.0
ARVERNE              Drinking                         859.0
ROSEDALE             Graffiti                         563.0
Name: Elapse Time (sec), Length: 775, dtype: float64
```

In [21]:

```python
# Visualize the average of Request_Closing_Time
avg_time = df.groupby('Complaint Type')['Elapse Time (sec)'].mean()
avg_time.plot(kind='barh')
plt.title("Average Response Time by Complaint Type")
plt.ylabel("Time (seconds)")
plt.show()
```



In [23]:
```
# Perform a Kruskal-WallisH test
# Fail to reject H0: All sample distributions are equal
# Reject H0: One or more sample distributions are not equal
groups = []
for complaint in df['Complaint Type'].unique():
    group = df[df['Complaint Type'] == complaint]['Elapse Time (sec)']
    groups.append(group)
stat, p_value = kruskal(*groups)
print("Statistic:", stat)
print("p-value:", p_value)
```

```
Statistic: 7655.370298880036
p-value: 0.0
```

Observations

1. The dataset contains information about customer service complaints including complaint type, city,etc. Some columns had missing values. Records with missing Closed Date were removed to obtain accurate time-based analysis. Missing values in City were replaced with “Unknown City” to avoid data loss and get clear result.
2. Some complaint types occur much more frequently than others.
3. Complaint distribution varies across cities, showing regional differences can face different issues.
4. Certain cities have higher complaint volumes compared to others.
5. The pattern of complaints differs by city, suggesting location-specific problems. Visualization (stacked bar chart) clearly shows variation in complaint types across cities.
6. The time taken to resolve complaints varies for different complaint types.
7. Some complaint types require significantly more time, indicating challenges or inefficiency.
8. Converting resolution time into seconds helped in better comparison and analysis. The average response time is not uniform across complaint types.
9. Some complaints are resolved quickly, while others take much more time. This indicates variation in complaint solving efficiency depending on complaint type.
10. Statistical Analysis (Kruskal-Wallis Test) p-value < 0.05 The Kruskal-Wallis test indicates a statistically significant difference in response times across complaint types. This means that at least one complaint type has a different resolution time compared to others.
11. The analysis highlights key complaint areas and performance gaps. It helps identify which complaint types need improvement in resolution time. These insights can be used to improve customer service efficiency and resource allocation.